Experiment 8 — Transactions and Isolation Levels

Aim

Implement banking-style transactions using BEGIN, COMMIT, ROLLBACK, and SAVEPOINT. Simulate partial failures. Demonstrate isolation levels and concurrency anomalies using two sessions.

Source Code

Setup — one connection

sql

```
-- ============================================================
-- Experiment 08 : Transactions and Isolation Levels
-- MySQL 8.0
-- ============================================================
DROP DATABASE IF EXISTS BankDB;
CREATE DATABASE BankDB;
USE BankDB;

CREATE TABLE Accounts (
    acc_no   INT PRIMARY KEY,
    holder   VARCHAR(50),
    balance  DECIMAL(12,2) NOT NULL
);

INSERT INTO Accounts VALUES
(1001,'Ravi' , 50000.00),
(1002,'Sita' , 30000.00),
(1003,'Arjun', 20000.00);

CREATE TABLE TxnLog (
    log_id   INT AUTO_INCREMENT PRIMARY KEY,
    action   VARCHAR(80),
    ts       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

);

Part A — COMMIT and ROLLBACK (single connection)

sql

```
-- A1: Successful transfer — COMMIT
START TRANSACTION;
UPDATE Accounts SET balance = balance - 5000 WHERE acc_no = 1001;
UPDATE Accounts SET balance = balance + 5000 WHERE acc_no = 1002;
INSERT INTO TxnLog(action) VALUES ('Transfer 5000: 1001 -> 1002');
COMMIT;

SELECT * FROM Accounts;

-- A2: Failed transfer — ROLLBACK
START TRANSACTION;
UPDATE Accounts SET balance = balance - 10000 WHERE acc_no = 1001;
-- simulate failure before credit
ROLLBACK;

SELECT * FROM Accounts;
```

Part B — SAVEPOINT with partial rollback

sql

```
START TRANSACTION;

UPDATE Accounts SET balance = balance - 2000 WHERE acc_no = 1001;
SAVEPOINT after_debit;

UPDATE Accounts SET balance = balance + 2000 WHERE acc_no = 1002;
SAVEPOINT after_credit;

-- simulate an error in a third step
```

UPDATE Accounts SET balance = balance + 2000 WHERE acc_no = 9999; -- no such account, 0 rows

SAVEPOINT after_failed_credit;

-- roll back only the failed step

ROLLBACK TO SAVEPOINT after_credit;

-- the first two updates survive

COMMIT;

SELECT * FROM Accounts;

Part C — Isolation levels (two-session demo)

Open **two query tabs** in dbfiddle (or two terminals). Copy Session A script into tab 1, Session B into tab 2, and execute in the order shown.

Demo 1 — READ UNCOMMITTED → dirty read

Session A (Tab 1):

sql

SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

START TRANSACTION;

UPDATE Accounts SET balance = 99999 WHERE acc_no = 1003;

-- do NOT commit yet

Session B (Tab 2) — run while A is still open:

sql

SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

START TRANSACTION;

SELECT balance FROM Accounts WHERE acc_no = 1003;

-- sees 99999 (dirty read)

COMMIT;

Session A — roll back:

sql

ROLLBACK;

Session B — re-check:

sql

SELECT balance FROM Accounts WHERE acc_no = 1003;

-- back to original value — proves the earlier read was dirty

Demo 2 — READ COMMITTED → non-repeatable read

Session A:

sql

SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

SELECT balance FROM Accounts WHERE acc_no = 1003;

Session B:

sql

SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

UPDATE Accounts SET balance = balance + 1000 WHERE acc_no = 1003;

COMMIT;

Session A — read again:

sql

SELECT balance FROM Accounts WHERE acc_no = 1003;

-- different value than the first read (non-repeatable read)

COMMIT;

Demo 3 — REPEATABLE READ → phantom read suppressed

Session A:

sql

SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;

START TRANSACTION;

SELECT COUNT(*) FROM Accounts WHERE balance > 25000;

Session B:

sql

INSERT INTO Accounts VALUES (1004, 'Meera', 40000.00);

Session A:

sql

```
SELECT COUNT(*) FROM Accounts WHERE balance > 25000;
-- same count as first SELECT (phantom suppressed)
COMMIT;
```

Demo 4 — SERIALIZABLE → full blocking

Session A:

sql

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
SELECT * FROM Accounts WHERE acc_no = 1001;
```

Session B (will block until A commits):

sql

```
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
UPDATE Accounts SET balance = balance + 500 WHERE acc_no = 1001;
-- waits for A's lock
```

Session A:

sql

```
COMMIT;
```

Session B now proceeds.

Cleanup — reset accounts

sql

```
UPDATE Accounts SET balance = 50000 WHERE acc_no = 1001;
UPDATE Accounts SET balance = 30000 WHERE acc_no = 1002;
UPDATE Accounts SET balance = 20000 WHERE acc_no = 1003;
DELETE FROM Accounts WHERE acc_no = 1004;
```

Output

Part	Query	Result
A1	SELECT * FROM Accounts after COMMIT	1001=45000, 1002=35000, 1003=20000
A2	SELECT * FROM Accounts after ROLLBACK	unchanged (45000, 35000, 20000)
B	SELECT * FROM Accounts	1001=43000, 1002=37000, 1003=20000
Demo 1	Session B read	**99999** — dirty read reproduced
Demo 2	Two reads in Session A	**different values** — non-repeatable read
Demo 3	Two counts in Session A	**same count** — phantom suppressed
Demo 4	Session B	**blocks** until A commits, then applies

Isolation-level anomaly matrix (MySQL 8 default = REPEATABLE READ):

Level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	✅ possible	✅ possible	✅ possible
READ COMMITTED	❌ prevented	✅ possible	✅ possible
REPEATABLE READ	❌ prevented	❌ prevented	❌ prevented (InnoDB gap locks)